MEDISOURCE AI

Hospital Procurement Automation & Intelligence Platform

This document details the architecture, design choices, microservices integration, and systemic fallbacks of the MediSource AI platform. Built specifically for the healthcare supply chain, the platform automates document ingestion, supplier onboarding, multi-level financial approvals, expiration-aware inventory management, contract lifecycle generation, and automated risk scoring.

Document Type:	Technical & Functional Architecture Guide
Target Domain:	Healthcare Supply Chain & Procurement Operations
Status:	Implementation Phase Complete (Docker Ready)
Created Date:	August 5, 2026
Platform Version:	v1.0.0 (FastAPI / React SPA)

1. Technology Stack in Layman's Terms

A modern enterprise web application has many moving parts. Rather than a single monolithic program, MediSource AI is built as a **Microservice Architecture** (meaning the application is split into 6 specialized mini-servers). Here is what each piece of technology does in simple terms:

Technology	Layman's Description & Role
FastAPI	The *engine room* of each microservice. It is a lightweight, super-fast framework that runs the server code and handles inputs and outputs (APIs).
React & Bootstrap	The *visual dashboard*. This is the web page that the hospital staff sees, clicks, and interacts with in their browser.
Nginx (API Gateway)	The *front-gate security guard*. It receives all incoming website traffic at http://localhost. It displays the React dashboard to users and routes any '/api/...' request to the correct microservice.
PostgreSQL	The *secured file rooms*. Under the 'Database-per-Service' rule, each service has its own separate database. Services never touch each other's database files directly, maintaining data isolation.
Apache Kafka	The *internal mail delivery system*. When a service finishes a task (like approving an invoice), it leaves a note (event) in Kafka. Other services listening for that note pick it up and do their job. This keeps services loosely coupled.
Temporal	The *executive assistant*. It handles long-running workflows that span days or weeks. If an invoice needs 3 managers to sign off, Temporal sends notifications, tracks deadlines, and escalates to the next manager if a deadline is missed.
Open Policy Agent (OPA)	The *official rulebook*. When an invoice comes in, OPA checks the hospital rules (e.g., 'If spend > $10,000, we need the CFO to approve') to decide the exact approval chain.
Redis	The *digital reservation sticky-notes*. It temporarily locks items (like reserving 100 boxes of gloves) while an order is being placed, preventing two departments from ordering the exact same physical items at the same instant.
MinIO	The *document warehouse*. An open-source, private version of Amazon S3. It stores actual files, such as PDFs of vendor invoices and finalized supply agreements.
scikit-learn & MLflow	The *artificial intelligence analyst*. It runs risk-prediction models on suppliers (e.g., probability of late delivery or fraud) and tracks model experiments in MLflow.

2. Microservices & Robust Fallbacks

Each of the six services functions as a specialized 'agent' focused on a single business domain. To ensure the system works reliably even when external components or databases fail, several fault-tolerance fallbacks have been built in:

A. Document Intelligence Service (Port 8001)

Role: Ingests PDF or image documents (invoices, purchase orders, quotes) uploaded by vendors or staff. It uses OpenCV to clean up scans (denoising, grayscaling, straightening) and PaddleOCR to extract written text into structured data (vendor name, invoice number, items, taxes, totals).

Fallback & Resilience: Text extraction is rarely 100% perfect. For every invoice, the service calculates an *average confidence score*. If the text extraction confidence falls below 75% or crucial fields (like 'Total Due') are missing, the system does not fail or reject the document. Instead, it flags the document for manual review and pushes it to a *Human-in-the-Loop Review Queue*. Hospital staff can then review the original side-by-side with the extraction and make manual corrections.

B. Vendor Intelligence Service (Port 8002)

Role: Manages onboarding and profiles for medical suppliers, pharmaceutical distributors, and equipment engineers. It validates tax IDs (GST), regulatory drug licenses, and CDSCO/FDA registrations.

Fallback & Resilience: The service contains a built-in validation rules engine. If an external license registry is offline or a watchlist lookup returns an error, the system does not crash or automatically block the vendor. Instead, it marks the validation as 'Under review' and moves the vendor profile to 'Pending Verification', notifying operators to manually approve or request further documents.

C. Approval Workflow Service (Port 8003)

Role: Governs the financial sign-off flow. It sends the purchase request details to Open Policy Agent (OPA), which calculates the required chain of human approvers (e.g. Ward Supervisor -> Department Head -> CFO). It then starts a Temporal workflow engine to track the sign-off steps.

Fallback & Resilience: If a hospital manager goes on vacation or forgets to click 'Approve', Temporal tracks the SLA deadline. On a timeout event (e.g., no action in 48 hours), Temporal triggers a fallback escalation mechanism, automatically rerouting the approval to the next-in-line supervisor or raising an alert. Redis is also used to prevent double-submissions, ensuring rapid double-clicks do not create duplicate workflows.

D. Inventory Intelligence Service (Port 8004)

Role: Tracks clinical stock. When a procurement request is approved, this service reserves stock units automatically. It is specifically built for healthcare: it separates inventory by *expiration dates* and *cold-chain requirements* (such as vaccines needing strict refrigeration).

Fallback & Resilience: To prevent two departments from reserving the exact same box of scarce medicine simultaneously, the service uses a *Redis Distributed Lock*. If the lock is held, a process will retry gracefully. If stock is unavailable, it automatically switches from a 'Reserved' status to a 'Backorder/Reorder' recommendation, sending notifications for replenishment without halting the system.

E. Contract Intelligence Service (Port 8005)

Role: Auto-populates Jinja2 contract templates (Supply Agreements, Equipment AMCs) using vendor and approval data. It converts these to signed digital contracts and pushes them to OpenSign.

Fallback & Resilience: OpenSign is an external e-signature service. If the OpenSign service is offline or fails to respond, the contract service falls back to a *Local Simulation Mode* (a fallback signature mock flow). This allows the testing and sandbox environments to progress with signature status updates immediately instead of failing. In production, failed webhook notifications are queued for retry in database state machines.

F. Risk Intelligence Service (Port 8006)

Role: Evaluates vendors using a Machine Learning gradient-boosted classifier. It predicts probability scores across five dimensions: Compliance, Fraud, Financial health, Supplier reliability, and Operational delivery. Experiments are logged directly into MLflow.

Fallback & Resilience: If the MLflow server is down or the pickled ML model file fails to load at startup, the service gracefully falls back to a *Deterministic Rule-Based Risk Engine*. It computes fallback scores using historical data points (e.g. number of late deliveries and days until license expiry), ensuring risk scoring remains active.

3. Connections & System Integration Flow

The microservices communicate using two distinct channels: **Synchronous API calls** (direct HTTP request/response for user interfaces) and **Asynchronous Event Pub/Sub** (via Apache Kafka for backend actions). Here is how a document travels through the system:

- **Step 1: Document Upload**: A user uploads an invoice. Nginx forwards the file to the Document Intelligence service. The file is saved in MinIO, OCR is run, and the extracted data is saved in PostgreSQL.
- **Step 2: Processed Event**: The Document Intelligence service publishes a *documents.processed* event to Kafka. The Vendor Intelligence and Approval Workflow services are listening for this event.
- **Step 3: Vendor & Approval Setup**: Vendor Intelligence updates the supplier profile. Approval Workflow uses OPA to determine the approval routing, writes a record to its PostgreSQL database, and starts a Temporal tracker.
- **Step 4: Human Approval**: The approver views the request on the React frontend (proxied via Nginx) and clicks 'Approve'. This signals the Temporal workflow, which fires an *approval.approved* event to Kafka.
- **Step 5: Reservation & Contract**: Inventory Intelligence consumes the approval event and reserves stock in PostgreSQL using a Redis lock. Contract Intelligence consumes the approval event, generates a contract document in MinIO, and triggers an e-signature envelope via OpenSign.
- **Step 6: Risk Analysis**: Risk Intelligence performs compliance/fraud scoring. If a contract is high-risk, it publishes a *risk.assessed* event to notify managers and block automated execution.

Architecture Summary: *This combination of direct HTTP gateways (Nginx) and decoupled event queues (Kafka) ensures that even if one service crashes (e.g., Risk Intelligence is temporarily down), users can still upload invoices and view dashboards. The system is resilient to individual service failures.*

4. Running the Codebase (Docker Setup)

To run this codebase locally without having to install and link Postgres, Kafka, OPA, and Temporal manually, you should use the provided Docker Compose files.

Prerequisites:

- **Docker Desktop:** Ensure Docker Desktop is installed and running on your Windows host.
- **Python:** Python 3.x on your PATH to run the mock seed data script.

Deployment Instructions:

1. Open a PowerShell terminal in the root directory.
2. Run the deployment script:

```
.\scripts\deploy.ps1
```

This script creates your config files, starts all 16 Docker containers, waits for core databases to become healthy, sets up the Kafka topics, runs SQL database migrations, and installs requests/reportlab libraries to inject dummy healthcare records.

3. Open your browser and navigate to **http://localhost** to interact with the frontend.

Console / Service Name	Address / Port	Description
Web Frontend Dashboard	http://localhost	Main UI dashboard for all microservices.
Temporal Dashboard	http://localhost:8088	Monitor approval workflows and timers.
MLflow Console	http://localhost:5000	Track ML training, parameters, and model registry.
MinIO Storage Console	http://localhost:9001	Explore raw uploaded PDFs and generated contracts.
OPA Engine REST	http://localhost:8181	Rules engine endpoints for evaluation.

End of Report